

# Trabajo Integrador – GESTION DE PAISES

## Alumnos:

Nicolas Cerdán – nicolas.e.cerdan@gmail.com

**Materia:** Programación 1

**Fecha de Entrega:** 29 de junio de 2026

## Índice

1. Introducción
2. Marco Teórico
3. Caso Práctico
4. Metodología Utilizada
5. Resultados Obtenidos
6. Conclusiones
7. Bibliografía
8. Anexos

## Marco Teórico

El presente trabajo se basa en los principios fundamentales de la **programación estructurada en Python**, aplicando conceptos de modularización, estructuras de datos y control de flujo para desarrollar un sistema de gestión de países.

Python es un lenguaje interpretado, multiparadigma y de alto nivel, que permite escribir código legible y reutilizable mediante funciones y módulos.

### ♦ Estructuras de datos

Se utilizaron **listas** y **diccionarios** como estructuras principales para almacenar y manipular la información de los países.

- Las **listas** permiten agrupar múltiples elementos y recorrerlos mediante bucles.
- Los **diccionarios** facilitan el acceso a los datos mediante claves, lo que mejora la organización y la eficiencia del programa.

### ♦ Modularización

El código se organizó en funciones independientes, cada una con una responsabilidad específica (agregar, buscar, filtrar, ordenar, mostrar estadísticas).

Esta práctica mejora la legibilidad, el mantenimiento y la reutilización del código, cumpliendo con el principio de “**divide y vencerás**”.

#### ♦ **Control de flujo y validaciones**

Se implementaron estructuras de control como `while`, `match`, `if/else` y manejo de excepciones con `try/except` para garantizar la estabilidad del programa.

El uso de `match` (introducido en Python 3.10) permite reemplazar largas cadenas de condiciones por una estructura más clara y ordenada.

El bloque `try/except` evita errores en tiempo de ejecución, controlando entradas inválidas o vacías.

#### ♦ **Buenas prácticas**

Se aplicaron convenciones de estilo según **PEP 8**, agregando comentarios descriptivos y validaciones para asegurar que el usuario ingrese datos correctos.

Además, se evitó el uso de variables globales, encapsulando la lista `países` dentro de la función `main()` para mantener la integridad de los datos.

## **Objetivo del Trabajo**

El objetivo principal de este proyecto integrador es **desarrollar un programa modular en Python** que permita gestionar información de países mediante operaciones de **búsqueda, filtrado, ordenamiento y análisis estadístico**.

El trabajo busca aplicar los conocimientos adquiridos en la materia **Programación I**, integrando conceptos de estructuras de datos, funciones, control de flujo y manejo de excepciones.

A través de este desarrollo, se pretende:

- Consolidar el uso de **listas y diccionarios** como estructuras dinámicas para almacenar información.
- Implementar un **menú interactivo** que facilite la navegación y ejecución de las distintas funciones del programa.
- Aplicar **validaciones y controles de errores** para garantizar la estabilidad y confiabilidad del sistema.
- Promover el uso de **buenas prácticas de programación**, como la modularización y la documentación del código.

- Favorecer la comprensión del proceso de diseño, prueba y depuración de un programa completo.

En síntesis, el trabajo tiene como propósito demostrar la capacidad de diseñar y construir una aplicación funcional que integre los fundamentos de la programación estructurada, utilizando Python como herramienta principal.

## Diseño del Caso Practico

- El caso práctico consiste en el desarrollo de un **programa de gestión de países** que permite realizar operaciones de búsqueda, filtrado, ordenamiento y análisis estadístico sobre una lista de países almacenada en memoria.

El diseño se basa en la **modularización del código**, dividiendo las tareas en funciones específicas para lograr claridad, reutilización y mantenimiento eficiente.

### ♦ Lógica general del programa

- El programa se estructura en torno a una función principal `main()`, que contiene el menú interactivo y la lista `países`.

Desde este punto se gestionan las distintas operaciones mediante funciones independientes:

- `agregar_pais(países)` → permite incorporar nuevos países con sus datos (nombre, población, superficie, continente).
- `buscar_pais(países)` → realiza búsquedas parciales o exactas por nombre.
- `filtrar_por_continente(países)` → muestra los países pertenecientes a un continente determinado.
- `ordenar_paises(países, criterio)` → ordena la lista según nombre, población o superficie.
- `mostrar_estadisticas(países)` → calcula promedios y muestra información general.

Cada función recibe la lista `países` como parámetro, evitando el uso de variables globales y garantizando la integridad de los datos.

### ♦ Estructura del menú

- El menú principal utiliza la instrucción `match` para simplificar la selección de opciones y mejorar la legibilidad del código.
- Se implementaron controles de entrada y manejo de errores con `try/except` para evitar fallos ante datos inválidos o vacíos.

Además, se agregó una validación que impide acceder a funciones de ordenamiento o filtrado si la lista `países` está vacía.

### ♦ Diagrama de flujo del programa

El flujo general del programa puede representarse de la siguiente manera:

Código

Inicio



main()



Mostrar menú principal



Seleccionar opción



Ejecutar función correspondiente



Volver al menú principal



Fin

### ♦ Estructuras de datos utilizadas

La información de cada país se almacena en **diccionarios**, y todos los diccionarios se agrupan dentro de una **lista**.

Ejemplo:

```
países = [  
    {"nombre": "Argentina", "poblacion": 46044703, "superficie": 2780400, "continente":  
    "América"},  
    {"nombre": "España", "poblacion": 47450795, "superficie": 505990, "continente": "Europa"}  
]
```

Esta estructura permite recorrer los datos fácilmente y aplicar filtros o ordenamientos según distintos criterios.

## Metodología Utilizada

- El desarrollo del proyecto se realizó siguiendo una metodología **iterativa y modular**, que permitió construir el programa paso a paso, verificando su funcionamiento en cada etapa y aplicando correcciones cuando fue necesario.

El proceso se organizó en distintas fases, respetando las buenas prácticas de programación y los criterios de la cátedra.

### ♦ Etapas del proceso

- **Análisis del enunciado**
  - Se interpretó la consigna del trabajo integrador para definir los requerimientos del programa: manejo de una lista de países, operaciones de búsqueda, filtrado, ordenamiento y estadísticas.
1. Se identificaron las funciones necesarias y los datos que debía contener cada país.
    - **Diseño de la estructura del programa**
    - Se planificó la estructura modular del código, definiendo una función principal `main()` y funciones auxiliares para cada operación.
  2. Se decidió utilizar una lista de diccionarios como estructura base para almacenar los datos de los países.
    - **Implementación del menú principal**
    - Se desarrolló un menú interactivo con la instrucción `match`, que permite seleccionar opciones de manera clara y ordenada.

3. Se agregaron controles de entrada y manejo de errores con `try/except` para evitar fallos ante datos inválidos.
  - **Desarrollo de funciones específicas**
  - Se implementaron las funciones de agregar, buscar, filtrar, ordenar y mostrar estadísticas, asegurando que todas reciban la lista `países` como parámetro para mantener la integridad de los datos.
4. Se añadieron validaciones para evitar el uso de opciones cuando la lista está vacía.
  - **Pruebas y depuración**
  - Se realizaron pruebas de cada función con distintos escenarios (entradas válidas, vacías, erróneas) para verificar el correcto funcionamiento.
5. Se corrigieron errores de lógica y se mejoraron los mensajes de salida para hacer el programa más claro y amigable.
  - **Documentación y comentarios**
  - Se agregaron comentarios detallados en cada parte del código para facilitar su comprensión.
6. Se elaboró el informe y el README siguiendo las pautas de la cátedra, incluyendo ejemplos y explicaciones del funcionamiento.
  -

## Resultados Obtenidos

El programa desarrollado cumple con los objetivos planteados, permitiendo gestionar información de países de manera eficiente y segura.

Durante las pruebas realizadas, se verificó el correcto funcionamiento de todas las funciones y la estabilidad del sistema ante distintos tipos de entrada.

### ◆ Funcionalidad general

El menú principal permite acceder de forma ordenada a cada operación del programa.

Las funciones responden adecuadamente a las acciones del usuario y muestran mensajes claros ante errores o entradas inválidas.

### ◆ Pruebas realizadas

Se realizaron pruebas con distintos escenarios para validar el comportamiento del programa:

| Escenario                                 | Resultado esperado                       | Resultado obtenido |
|-------------------------------------------|------------------------------------------|--------------------|
| <b>Agregar país con datos válidos</b>     | País agregado correctamente a la lista   | ✓ Correcto         |
| <b>Buscar país por nombre parcial</b>     | Se muestran coincidencias                | ✓ Correcto         |
| <b>Buscar país con campo vacío</b>        | Mensaje de error y nueva solicitud       | ✓ Correcto         |
| <b>Filtrar por continente inexistente</b> | Mensaje “No se encontraron países”       | ✓ Correcto         |
| <b>Ordenar por población</b>              | Lista ordenada de menor a mayor          | ✓ Correcto         |
| <b>Intentar ordenar lista vacía</b>       | Mensaje de advertencia y retorno al menú | ✓ Correcto         |

### ♦ Validaciones y controles

Se implementaron controles para evitar errores comunes:

- Verificación de entradas vacías o numéricas en búsquedas y filtros.
- Manejo de excepciones con `try/except` para prevenir interrupciones del programa.
- Control de lista vacía antes de ejecutar funciones de ordenamiento o filtrado.
- Mensajes informativos para guiar al usuario en cada paso.

### ♦ Evaluación general

El programa demostró ser estable, modular y fácil de usar.

Las validaciones implementadas garantizan que el usuario no pueda ingresar datos erróneos ni ejecutar operaciones sin información cargada.

El código está completamente comentado, lo que facilita su comprensión y mantenimiento.

## Conclusiones

El desarrollo de este trabajo integrador permitió aplicar de manera práctica los conceptos fundamentales de la programación estructurada en Python, consolidando conocimientos sobre estructuras de datos, modularización, control de flujo y manejo de excepciones.

A lo largo del proceso se logró construir un programa funcional, claro y estable, capaz de gestionar información de países mediante operaciones de búsqueda, filtrado, ordenamiento y análisis estadístico.

### ♦ Aprendizajes obtenidos

- Se comprendió la importancia de **organizar el código en funciones** para mejorar la legibilidad y facilitar el mantenimiento.
- Se afianzó el uso de **listas y diccionarios** como estructuras dinámicas para almacenar y manipular datos.
- Se adquirió experiencia en la implementación de **validaciones y manejo de errores** con `try/except`, garantizando la robustez del programa ante entradas inválidas.
- Se reforzó la práctica de **documentar el código** con comentarios claros y precisos, siguiendo las recomendaciones de la cátedra.

### ♦ Importancia de las herramientas utilizadas

Las herramientas y estructuras empleadas son esenciales para el desarrollo de software confiable y escalable.

El uso de `match` simplificó la lógica del menú, mientras que las validaciones y controles de flujo permitieron crear una experiencia de usuario más segura y ordenada.

La modularización y el paso de parámetros entre funciones demostraron cómo mantener la integridad de los datos sin recurrir a variables globales.



## ♦ Reflexión final

Este trabajo integrador no solo permitió aplicar los contenidos teóricos de la materia, sino también desarrollar habilidades de **análisis, diseño y resolución de problemas**.

El proceso de prueba y depuración fue clave para comprender la importancia de la planificación y la mejora continua en el desarrollo de programas.

En definitiva, el proyecto representa una síntesis del aprendizaje alcanzado en Programación I, evidenciando la capacidad de transformar conceptos teóricos en soluciones prácticas y funcionales.

## Bibliografía

- [Python Software Foundation. \(2024\). \*Python Documentation\*.  
https://docs.python.org/3/tutorial/](https://docs.python.org/3/tutorial/)
- [Sweigart, A. \(2023\). \*Automate the Boring Stuff with Python\*. No Starch Press.](#)
- [UTN FRSN – Cátedra de Programación I \(2025\). \*Tutorial Proyecto Integrador\*.](#)
- 

## Anexos

- **Python 3.12** como lenguaje de programación.
- **Visual Studio Code** para la edición y prueba del código.
- **GitHub** para el repositorio y control de versiones.
- **Google Docs** para la redacción del informe.